Department of Computer Science

Course Code: CIC201

Course Title: Artificial Intelligence

Fall 2025

Lab 09

Linear Regression & Decision Tree

Objective:

This experiment introduces the students to the basics of Linear Regression and Decision Tree models. The lab explains the induction of a decision tree. It also discusses how regression can be used to make predictions on different data sets and used as a tool for the Supervised form of Artificial Intelligence.

Name of Student	
Student ID	
Date of Lab Conducted	
Marks Obtained	
Remarks	
Signature	

1. Objective

This experiment introduces the students to the basics of Linear Regression and Decision Tree models. The lab explains the induction of a decision tree. It also discusses how regression can be used to make predictions on different data sets and used as a tool for the Supervised form of Artificial Intelligence.

.

2. Instructions

You have to perform the following tasks yourselves. Raise your hand if you face any difficulty in understanding and solving these tasks. Plagiarism is an abhorrent practice and you should not engage in it.

3. How to Submit

Submit lab work using TEAMS

Linear Regression

There are two types of supervised machine learning algorithms: Regression and classification. The former predicts continuous value outputs while the latter predicts discrete outputs. For instance, predicting the price of a house in dollars is a regression problem whereas predicting whether a tumor is malignant or benign is a classification problem.

Regression models a target prediction value based on independent variables. It is mostly used for finding out the relationship between variables and forecasting. There are different regression models based on the kind of relationship between dependent and independent variables, and the number of independent variables being used.

Uni-variate and Multivariate:

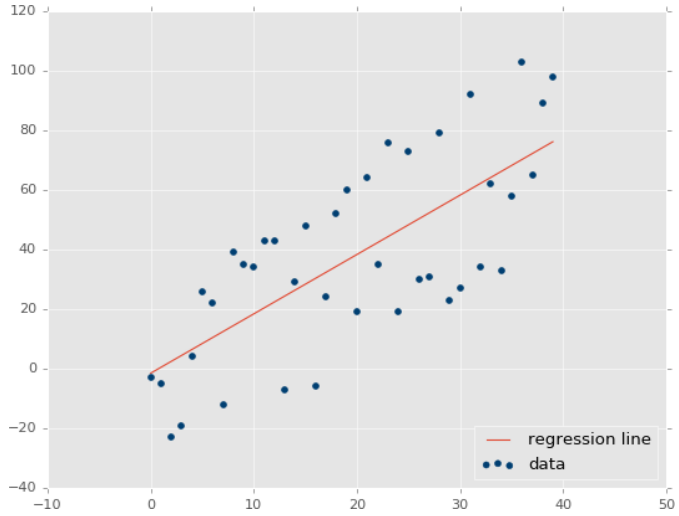
When there is only feature it is called **Uni-variate** Linear Regression and if there are multiple features, it is called **Multiple** Linear Regression.

Remember, a linear regression model in two dimensions is a straight line; in three dimensions it is a plane, and in more than three dimensions, a hyperplane.

Linear Regression

The term "linearity" in algebra refers to a linear relationship between two or more variables. Linear regression performs the task to predict a dependent variable value (y) based on a given independent variable (x). So, this regression technique finds out a linear relationship between x (input) and y (output). Hence, the name is Linear Regression. If we plot the independent variable

(x) on the x-axis and dependent variable (y) on the y-axis, linear regression gives us a straight line that best fits the data points, as shown in the figure below.



s

The equation of the above line is:

Y= mx + b

Where b is the intercept and m is the slope of the line. So basically, the linear regression algorithm gives us the most optimal value for the intercept and the slope (in two dimensions). The y and x variables remain the same, since they are the data features and cannot be changed. The values that we can control are the intercept(b) and slope(m). There can be multiple straight lines depending upon the values of intercept and slope. Basically, what the linear regression algorithm does is, it fits multiple lines on the data points and returns the line that results in the least error.

Linear regression is a very powerful technique and can be used to understand the factors that influence profitability. It can be used to forecast sales in the coming months by analyzing the sales data for previous months. It can also be used to gain various insights about customer behavior.

Let us apply linear regression on an example dataset as shown below:

1	YearsExperience	Salary
2	1.1	39343
3	1.3	46205
4	1.5	37731
5	2	43525
6	2.2	39891
7	2.9	56642
8	3	60150
9	3.2	54445
10	3.2	64445
11	3.7	57189
12	3.9	63218
13	4	55794
14	4	56957
15	4.1	57081
16	4.5	61111
17	4.9	67938
18	5.1	66029
19	5.3	83088
20	5.9	81363

Importing all the required libraries:

```
import numpy as np
import matplotlib.pyplot as plot
import pandas
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics
```

The next step is to import the data and divide it into “attributes” and “labels”. Attributes are the independent variables while labels are dependent variables whose values are to be predicted. In our dataset, we only have two columns. We want to predict the Salary depending upon the Year of experience recorded. Therefore, our attribute set will consist of the “YearExperience” column which is stored in the x variable, and the label will be the “Salary” column which is stored in y variable. The following command imports the dataset using pandas:

```
# Import the dataset
dataset = pandas.read_excel('SalaryAge.xlsx')
x = dataset['YearsExperience'].values.reshape(-1,1)
y = dataset['Salary'].values.reshape(-1,1)
```

let's plot our data points on a 2-D graph to look at our dataset and see if we can manually find any relationship between the data using the below script:

```
dataset.plot(x='YearsExperience', y='Salary', style='o')
plot.title('YearsExperience vs Salary')
plot.xlabel('YearsExperience')
plot.ylabel('Salary')
plot.show()
```



Next, we split 80% of the data to the training set while 20% of the data to test set using below code. The test_size variable is where we actually specify the proportion of the test set.

```
# Split the dataset into the training set and test set
xTrain, xTest, yTrain, yTest = train_test_split(x, y, test_size = 0.2, random_state = 0)
```

After splitting the data into training and testing sets, finally, the time is to train our algorithm. For that, we need to import LinearRegression class, instantiate it, and call the fit() method along with our training data.

```
linearRegressor = LinearRegression()
linearRegressor.fit(xTrain, yTrain)
```

Now that we have trained our algorithm, it's time to make some predictions. To do so, we will use our test data and see how accurately our algorithm predicts the percentage score. To make predictions on the test data, execute the following script:

```
yPrediction = linearRegressor.predict(xTest)
```

Now compare the actual output values for X_test with the predicted values, execute the following script:

```
df = pandas.DataFrame({'Actual': yTest.flatten(), 'Predicted': yPrediction.flatten()})
df
```

	Actual	Predicted
0	37731	40748.961841
1	122391	122699.622956
2	57081	64961.657170
3	63218	63099.142145
4	116969	115249.562855
5	109431	107799.502753

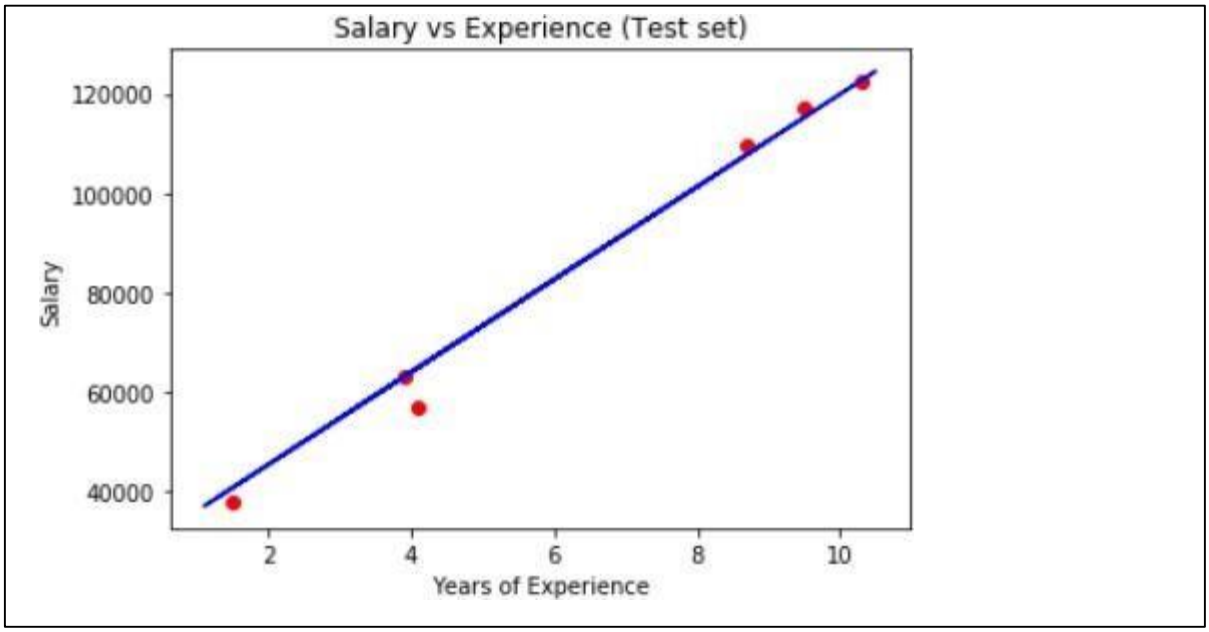
Let's plot our regression line with training data:

```
plot.scatter(xTrain, yTrain, color = 'red')
plot.plot(xTrain, linearRegressor.predict(xTrain), color = 'blue')
plot.title('Salary vs Experience (Training set)')
plot.xlabel('Years of Experience')
plot.ylabel('Salary')
plot.show()
```



Now plot it with test data:

```
plot.scatter(xTest,yTest,color='red')
plot.plot(xTest, linearRegression.predict(xTest), color='blue')
plot.title("salary vs Exp (Training set)")
plot.xlabel("years of exp")
plot.ylabel("salary")
plot.show()
```



The final step is to evaluate the performance of the algorithm. This step is particularly important to compare how well different algorithms perform on a particular dataset. For regression algorithms, three evaluation metrics are commonly used:

1. **Mean Absolute Error** (MAE) is the mean of the absolute value of the errors. It is calculated as:

$$MAE = \frac{1}{n}\sum_{j=1}^{n}|y_j - y_j|$$

Mean Absolute Error

2. **Mean Squared Error** (MSE) is the mean of the squared errors and is calculated as:

$$MSE = \frac{1}{N}\sum_{i}^{n}(Y_i - y_i)^2$$

Mean Squared Error

3. Root Mean Squared Error (RMSE) is the square root of the mean of the squared errors:

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{j=1}^{n}(y_j - \hat{y}_j)^2}$$

Root Mean Squared Error

The Scikit-Learn library comes with pre-built functions that can be used to find out these values for us.

Let us find the values for these metrics using our test data.

```
print('Mean Absolute Error:', metrics.mean_absolute_error(yTest, yPrediction))
print('Mean Squared Error:', metrics.mean_squared_error(yTest, yPrediction))
print('Root Mean Squared Error:', np.sqrt(metrics.mean_squared_error(yTest, yPrediction)))
```

```
Mean Absolute Error: 2446.1723690465064
Mean Squared Error: 12823412.298126562
Root Mean Squared Error: 3580.979237321345
```

The higher values of the error shows that the algorithm is not very accurate but can still make predictions.

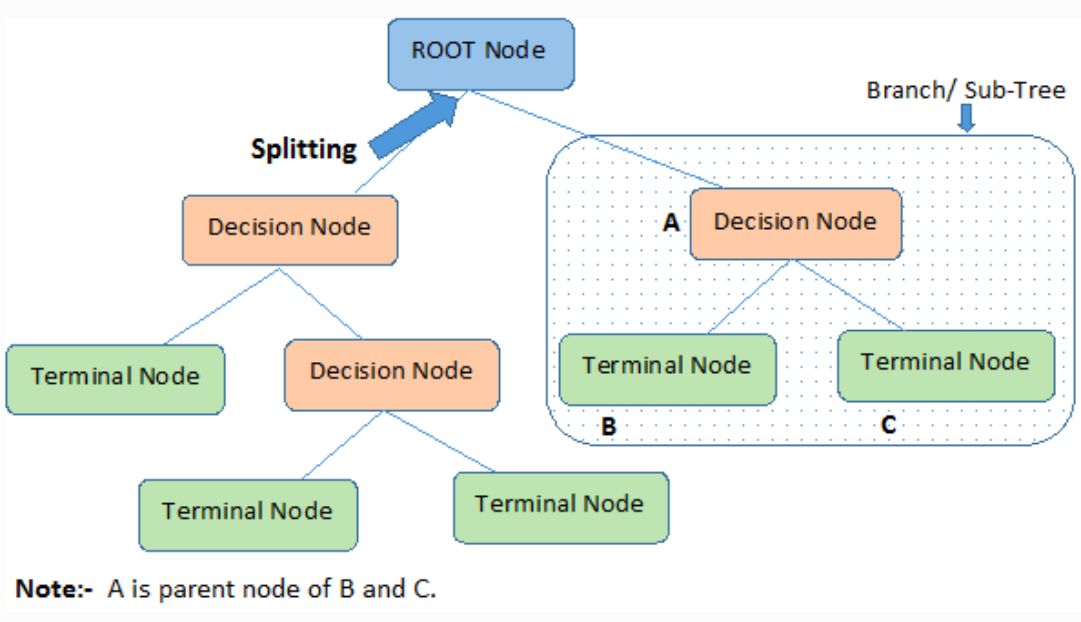
Decision Tree

A decision tree is a structure that can be used to divide up a large collection of records into successively smaller sets of records by applying a sequence of simple decision rules. With each successive division, the members of the resulting sets become more and more similar to one another.

A decision tree model consists of a set of rules for dividing a large heterogeneous population into smaller, more homogeneous groups with respect to a particular target variable. The target variable is usually categorical and the decision tree model is used either to calculate the probability that a given record belongs to each of the categories, or to classify the record by

assigning it to the most likely class. Decision trees can also be used to estimate the value of a continuous variable, although there are other techniques more suitable to that task.

Important Decision Tree Terms



A record enters the tree at the root node. The root node applies a test to determine which child node the record will encounter next. There are different algorithms for choosing the initial test, but the goal is always the same: To choose the test that best discriminates among the target classes. This process is repeated until the record arrives at a terminal node. All the records that end up at a given terminal of the tree are classified the same way. There is a unique path from the root to each leaf. That path is an expression of the rule used to classify the records.

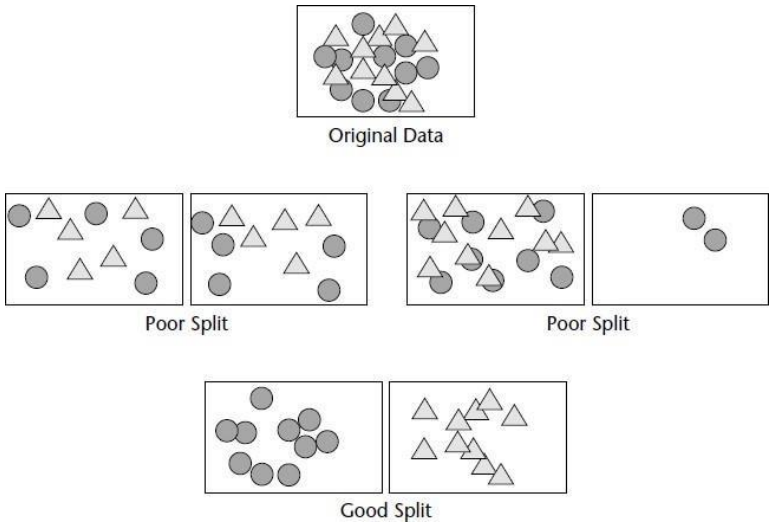
How a Decision Tree Is Grown

Although there are many variations on the core decision tree algorithm, all of them share the same basic procedure: Repeatedly split the data into smaller and smaller groups in such a way that each new generation of nodes has greater purity than its ancestors with respect to the target variable. For most of this discussion, we assume a binary, categorical target variable. At the start of the process, there is a training set consisting of preclassified records—that is, the value of the target variable is known for all cases. The goal is to build a tree that assigns a class (or a likelihood of membership in each class) to the target field of a new record based on the values of the input variables.

The tree is built by splitting the records at each node according to a function of a single input field. The first task, therefore, is to decide which of the input fields makes the best split. The best

split is defined as one that does the best job of separating the records into groups where a single class predominates in each group.

The measure used to evaluate a potential split is purity.



Measures of Calculating the Purity

A number of different measures are available to evaluate potential splits. Algorithms developed in the machine learning community focus on the increase in purity resulting from a split, while those developed in the statistics community focus on the statistical significance of the difference between the distributions of the child nodes. Alternate splitting criteria often lead to trees that look quite different from one another, but have similar performance. That is because there are usually many candidate splits with very similar performance. Different purity measures lead to different candidates being selected, but since all of the measures are trying to capture the same idea, the resulting models tend to behave similarly

Purity measures for evaluating splits for categorical target variables include:

- Gini (also called population diversity)
- Entropy (also called information gain)
- Information gain ratio
- Chi-square test

Growing the Full Tree

The initial split produces two or more child nodes, each of which is then split in the same manner as the root node. Once again, all input fields are considered as candidate splitters, even fields already used for splits. However, fields that take on only one value are eliminated from

consideration since there is no way that they can be used to create a split. A categorical field that has been used as a splitter higher up in the tree is likely to become single-valued fairly quickly. The best split for each of the remaining fields is determined. When no split can be found that significantly increases the purity of a given node, or when the number of records in the node reaches some preset lower bound, or when the depth of the tree reaches some preset limit, the split search for that branch is abandoned and the node is labeled as a leaf node.

Student Exercise

Task 1

Implement the examples given in the manual and elaborate the results.

Task 2

Apply linear regression on the dataset provided with the manual (weather.xlsx) and predict the minimum temperature based on the maximum temperature (Two Columns Only). Elaborate the results.

Task 3

For the heart disease dataset (heart.csv), use the categorical variable to learn a decision tree to classify the records. You model should be learnt against a 70-30 testing training split. All the performance evaluation matrices should be reported.

Task 4

Explore the Weather_Notebook_Kaggle provided to you along with the lab material.